Aptitude Arena

Design and System Architecture of a Full-Stack Web Platform for Competitive Examination Preparation

Aptitude Arena: Design and System Architecture of a Full-Stack Web Platform for Competitive Examination Preparation

Krishna Veer Singh, Kushagra Garg, Kartik, Mudit Sharma, and Piyush Kumar
Team EMSQUAD — Project Code: PBL2627-CSE-B-037

Abstract

Competitive examination preparation for high-stakes assessments such as JEE, SSC, GATE, and RAS remains fragmented across disconnected question repositories, unverified practice material, and inconsistent evaluation practices, leaving aspirants without an integrated environment for authoritative practice and longitudinal self-assessment. This paper presents **Aptitude Arena**, a full-stack web platform designed to consolidate examination discovery, timed assessment, server-side evaluation, and progress analytics within a single secure ecosystem. The system adopts a decoupled, layered client-server architecture comprising a responsive frontend interface, a Spring Boot REST API layer organized into controller, service, and repository tiers, and a normalized PostgreSQL relational persistence layer enforcing ACID transactions and strict referential integrity. A defining design decision is the server-side evaluation model: answer keys are masked during live attempts to prevent client-side inspection or reverse engineering, and all scoring—including negative-marking penalties—is computed authoritatively on the backend through a marking-engine algorithm. Security is addressed through stateless JWT authentication, role-based access control separating student capabilities from administrative CRUD controls, BCrypt password hashing, and alignment with the OWASP API Security Top 10. The platform targets a sub-500 ms 95th-percentile response time (p_{95}) under concurrent exam loads, achieved through database indexing, and adheres to WCAG 2.2 accessibility guidelines for high-contrast rendering and keyboard navigation. Pedagogically, the architecture operationalizes retrieval practice (the testing effect) and distributed (spaced) learning by transforming previous-year question sets into active, interval-based evaluation interfaces with post-submission explanatory feedback. Aptitude Arena demonstrates how a layered full-stack architecture can deliver secure, scalable, and pedagogically grounded examination preparation at scale.

Index Terms

Competitive Examination Preparation; Full-Stack Web Platform; Spring Boot; RESTful API; PostgreSQL; Server-Side Evaluation and Scoring; Retrieval Practice; Spaced Learning; Role-Based Access Control; JWT Authentication; ACID Transactions.

A few notes on the choices I made, Mudit: I kept the abstract to IEEE's conventional single-paragraph form (~240 words) and pushed all implementation detail—schema, endpoint tables, functional requirements—out of the abstract so it functions as a genuine front-matter summary rather than a condensed design document. The three contributions the abstract foregrounds are (1) the consolidated discovery-to-analytics pipeline, (2) the server-side answer-key masking and marking engine, and (3) the mapping of cognitive-science principles onto the platform's mechanics. The title and author block are included above so the front matter reads as a coherent unit; if you'd like, I can draft a matching **Introduction** section next, or compress this abstract to a strict 150-word IEEE short-format limit.

1. Introduction and Problem Statement

1.1 Background and Motivation

Competitive examinations function as high-stakes gateways to higher education and public-sector employment in India, and the scale of candidature they attract makes them a defining feature of the national assessment landscape. The Joint Entrance Examination governs admission to premier engineering institutions, the Graduate Aptitude Test in Engineering serves as the qualifying benchmark for postgraduate engineering study and public-sector recruitment, the Staff Selection Commission examinations determine selection into a broad range of central government posts, and the Rajasthan Administrative Service examination regulates entry into the state civil services. Although these examinations differ in syllabus, question format, and scoring logic, they share a common structural demand: candidates must demonstrate fluent, durable recall across multiple subjects under severe time constraints and, in most cases, under negative-marking regimes that penalize incorrect responses [1]. Preparation is therefore not merely a matter of content exposure but of repeated, timed, and accurately evaluated practice.

The dominant contemporary response to this demand has been digital. A substantial body of engineering literature has examined online examination systems, generally framed as institutional assessment instruments rather than as learner-directed preparation environments. Govind Raj, for example, presented an architectural design for a national online examination system using the Flex, Spring, and Hibernate frameworks, with dynamic paper generation based on item response theory and a design intended to support large concurrent loads through cloud computing [2]. Oppl and colleagues proposed a flexible platform for computerized adaptive testing that separates the item-management core from the test engine so that domain and strategy components can be exchanged without modifying the core implementation [3]. Other work has described browser/server examination portals for automated grading [4], client/server architectures emphasizing dataset design and reduced resource consumption [5], and cloud-based solutions for managing examination resources in educational institutions [6]. These contributions establish that the architectural primitives for secure, scalable online assessment are well understood. What they largely leave unaddressed is the *preparation* problem: the sustained, self-directed, longitudinal workflow by which an aspirant discovers an examination, drills its previous-year questions, receives trustworthy feedback, and revisits material across time.

Aptitude Arena is proposed to occupy precisely this gap. It is a full-stack web platform that consolidates examination discovery, filtered question practice, timed assessment, server-side evaluation, and progress review within a single application, and it is designed so that its mechanics reflect established findings from cognitive and educational psychology rather than treating practice as an undifferentiated activity.

1.2 The Fragmentation Problem in Competitive Examination Preparation

The preparation workflow of a typical aspirant for JEE, SSC, GATE, or RAS is distributed across a heterogeneous and largely uncoordinated set of resources. Syllabus references are consulted in official notifications or third-party summaries; previous-year question papers are obtained from scattered repositories, scanned compilations, or social-media groups; practice is conducted on generic quiz sites that do not distinguish between examination patterns; explanations for incorrect answers are sought separately, often from community forums of uncertain reliability; and progress is tracked in ad hoc notebooks or not at all. Each of these fragments carries a distinct failure mode.

First, **discovery and scoping are manual and error-prone**. Because question pools are rarely organized by examination, subject, and year in a consistent taxonomy, the act of narrowing practice to "GATE Computer Science, 2021, Discrete Mathematics" requires the candidate to navigate multiple disconnected sources before a single question is attempted.

Second, **evaluation is frequently untrustworthy or client-exposed**. Systems that compute scores or reveal answer keys on the client side are vulnerable to inspection and reverse engineering, and any platform intended for serious preparation must ensure that scoring logic and correct answers cannot be manipulated or harvested from the browser. The literature on examination-system architecture has consistently emphasized secure, server-mediated evaluation and carefully planned data models as prerequisites for institutional credibility [2], [5], yet many consumer-facing practice tools do not implement these guarantees.

Third, **feedback is decoupled from practice**. Where a candidate answers a question incorrectly, the corrective information—the correct option, the reasoning, the associated concept—is often absent or delivered only as a terminal score. This severs the retrieval event from the feedback that makes it instructive, undermining the very mechanism that gives practice its pedagogical value.

Fourth, **no longitudinal record is preserved**. Because practice, feedback, and progress live in separate places, the candidate accumulates no durable activity history. Without such a record, distributed review—returning to previously attempted questions at intervals—is effectively impossible to schedule or verify.

These four failures compound one another: fragmentation at the discovery layer produces inconsistent question pools; client-exposed evaluation produces unreliable scoring; decoupled feedback produces weak learning events; and the absence of a longitudinal record prevents the spaced revisitation on which retention depends. The result is a preparation experience that is effortful but unmeasured, and in which the learner cannot easily determine whether practice is producing durable competence or merely transient familiarity.

1.3 Cognitive Foundations: Retrieval Practice and Distributed Practice

The design rationale for a unified practice platform is grounded in two of the most robustly supported findings in the cognitive psychology of learning, both of which the platform is intended to operationalize.

Retrieval practice. Roediger and Karpicke demonstrated that taking a memory test does not merely assess what a learner knows but actively enhances later retention—a phenomenon known as the *testing effect* [7]. In their experiments, students who studied prose passages and then took one or three immediate free-recall tests, without feedback, were compared against students who restudied the material an equivalent number of times. When the final retention test was administered after five minutes, repeated studying produced better recall than repeated testing; but on the delayed tests administered after two days and one week, prior testing yielded substantially greater retention than studying, even though repeated studying had inflated students' confidence in their ability to remember the material [7]. Two features of this result are consequential for system design. The first is that the *benefit of retrieval is latent*—it appears on delayed, not immediate, measures—which means a preparation platform must be built to support revisitation over time, not simply to deliver a score at the end of an attempt. The second is the *metacognitive illusion* it exposes: because restudy feels more productive in the moment, learners tend to choose it over testing, and a platform that merely offers both will not automatically steer users toward the more effective strategy [7], [8].

The retrieval-practice effect has since been corroborated at scale. Rowland's meta-analysis of testing versus restudy found support for effortful processing as a contributor to the testing effect, with initial recall tests yielding larger testing benefits than recognition tests [9]. Yang and colleagues integrated data from 48,478 students drawn from 222 independent studies and found that testing and quizzing raised classroom academic achievement to a medium extent, with an overall effect size of $g = 0.499$; the magnitude of the effect was modulated by factors including the provision of corrective feedback, test format consistency, material matching, and the number of test repetitions [10]. The feedback and repetition moderators are directly actionable: they indicate that a platform should not only administer tests but should return corrective feedback after submission and should make repeated encounters with the same material easy to initiate.

Distributed practice. Dunlosky and colleagues reviewed ten widely used learning techniques and assessed their relative utility across four categories of variables—learning conditions, student characteristics, materials, and criterion tasks [1]. Only two techniques received a *high utility* assessment: practice testing and distributed practice. These two were distinguished because their benefits generalize across learners of different ages and abilities and have been shown to boost performance across many criterion tasks and in genuine educational contexts. By contrast, five techniques—summarization, highlighting, the keyword mnemonic, imagery use for text learning, and rereading—received low utility ratings, in part because they do not consistently improve performance despite their popularity among students [1]. The same conclusion has been echoed in subsequent reviews, which note that practice testing and distributed practice are the techniques that merit a high utility rating precisely because they benefit diverse learners across a variety of tasks and contexts [11], and a meta-analytic follow-up has since quantified the effect sizes that Dunlosky's qualitative ratings implied [12].

The spacing literature reinforces the point. Kang, summarizing hundreds of studies in cognitive and educational psychology, observed that spacing out repeated encounters with material over time produces superior long-term learning compared with repetitions that are massed together, and that incorporating tests into spaced practice amplifies the benefits; spaced review was found to enhance diverse forms of learning, including memory, problem solving, and generalization to new situations [13]. Roediger and Pyc identified distribution of practice, retrieval practice through self-testing, and explanatory questioning as three inexpensive, empirically supported principles applicable from kindergarten through higher education, noting that these techniques are nonetheless

rarely applied in practice and are frequently supplanted by instructional habits known to be inferior [14]. Soderstrom, Kerr, and Bjork further demonstrated that retrieval and spacing interact—repeated testing and restudying each improved learning once spacing differences were controlled—indicating that the two mechanisms should be designed in concert rather than treated as independent features [15].

Taken together, this literature yields three design imperatives. Practice must be *active retrieval* rather than passive review; it must be *distributed over time* rather than massed; and it must be accompanied by *corrective feedback* to be maximally effective. A platform that merely hosts a question bank satisfies none of these imperatives automatically—it must be architected so that retrieval is a first-class workflow, revisitation is frictionless, and feedback is delivered reliably after submission.

1.4 Problem Statement

In light of the fragmentation described in Section 1.2 and the pedagogical requirements established in Section 1.3, the problem addressed by this work can be stated as follows:

> There is no unified, secure, and pedagogically grounded platform that allows competitive-examination aspirants across JEE, SSC, GATE, and RAS to discover examinations, narrow question pools by subject and year, attempt previous-year questions under timed conditions, receive trustworthy server-side evaluation with post-submission feedback, and review their performance longitudinally—while simultaneously providing administrators a controlled means to maintain examination metadata and question content with integrity.

Existing online examination systems are predominantly oriented toward institutional assessment and stop short of the learner-directed preparation workflow [2], [3], [4], [6]. Consumer practice tools, conversely, frequently compute scores or expose answer keys on the client, decouple feedback from the retrieval event, and preserve no durable activity record. Neither category integrates the cognitive-science findings on retrieval practice and distributed practice into its mechanics. The specific deficiencies to be resolved are therefore: (i) the absence of a consolidated discovery-to-practice pipeline; (ii) the absence of server-authoritative evaluation that masks answer keys during live attempts and computes marking, including negative-marking penalties, on the backend; (iii) the absence of a longitudinal attempt-and-response record that makes distributed review feasible; and (iv) the absence of a design that maps evidence-based learning principles onto concrete platform features.

1.5 Research Objectives and Contributions

This paper pursues a design-oriented methodology: it derives requirements from the project specification and interface prototype, defines the system architecture and data relationships, specifies representative API contracts, and proposes a verification plan. Its principal contributions are:

1. **A unified preparation architecture.** A decoupled, layered full-stack design that links examination discovery to filtered practice, timed assessment, server-side evaluation, and progress analytics within a single ecosystem, eliminating the fragmentation characteristic of current workflows.

2. **A server-authoritative evaluation model.** An architecture in which answer keys are masked during live attempts and all scoring, including negative-marking penalties, is computed on the backend through a marking-engine algorithm, with security hardened via stateless authentication, role-based access control, and adherence to recognized API-security guidance.

3. **A data model supporting longitudinal review.** A normalized relational schema—covering users, examinations, subjects, questions, answer options, tests, attempts, and responses, with supporting structures for bookmarks and daily activity—that preserves the activity history required for distributed practice.

4. **An explicit mapping of learning science to system mechanics.** A design rationale that operationalizes retrieval practice and distributed practice as first-class features (topic-based quizzes, reviewable attempts, and planned revisitation), while maintaining the epistemically appropriate caveat that the cited literature supports the educational rationale for such features but does not by itself demonstrate that this specific platform improves examination scores [1], [7], [13].

1.6 Organization of the Paper

The remainder of this paper is organized as follows. Section 2 reviews related work in online examination system architecture and the cognitive psychology of learning techniques, positioning Aptitude Arena against existing contributions. Section 3 presents the system architecture and design, including the layered backend, the

frontend interface, and the security model. Section 4 specifies the data model and its relationships. Section 5 defines representative API contracts and the server-side marking engine. Section 6 describes the verification and evaluation plan. Section 7 concludes the paper and outlines directions for future work.

Mudit, a few notes on how I've framed this: the section follows the conventional IEEE introduction arc—background, the specific problem, the literature that motivates the design, a formal problem statement, enumerated contributions, and a roadmap—and it integrates Roediger & Karpicke (2006) and Dunlosky et al. (2013) not as decorative citations but as the source of the three design imperatives (active retrieval, distribution over time, corrective feedback) that the later architecture sections must satisfy. I also preserved the epistemic caveat from your own draft: the psychology literature justifies the *rationale* for these features but does not prove a marks improvement, which is important for reviewer credibility. If you'd like, I can next draft **Section 2**, or compress this introduction to a tighter two-page version.

2. Proposed System Architecture

2.1 Architectural Overview

Aptitude Arena is specified as a decoupled, layered three-tier architecture that separates the browser interface, the application logic, and the persistent data store. The design rationale for this separation is twofold: it permits the frontend and backend to be developed, tested, and deployed independently while retaining a stable contract for data exchange, and it concentrates authority over scoring and answer keys in the server tier where it cannot be inspected or manipulated by the client [16]. The three tiers are connected by two well-defined boundaries — an HTTPS/REST-JSON boundary between the presentation tier and the application tier, and a JDBC/ACID-transaction boundary between the application tier and the data tier — so that no tier reaches directly into another's internal representation.

The proposed implementation realizes the presentation tier as a React application written in TypeScript, the application tier as a Java Spring Boot service exposing RESTful endpoints, and the data tier as a PostgreSQL relational database. The backend stack of Java, Spring Boot, and PostgreSQL, together with Maven as the build tool, follows the project specification [16], and the three-tier arrangement echoes the client/server separation advocated in prior online-examination architecture work, where the deliberate planning of the data layer and the choice of an open-source stack are treated as prerequisites for a maintainable system [5].

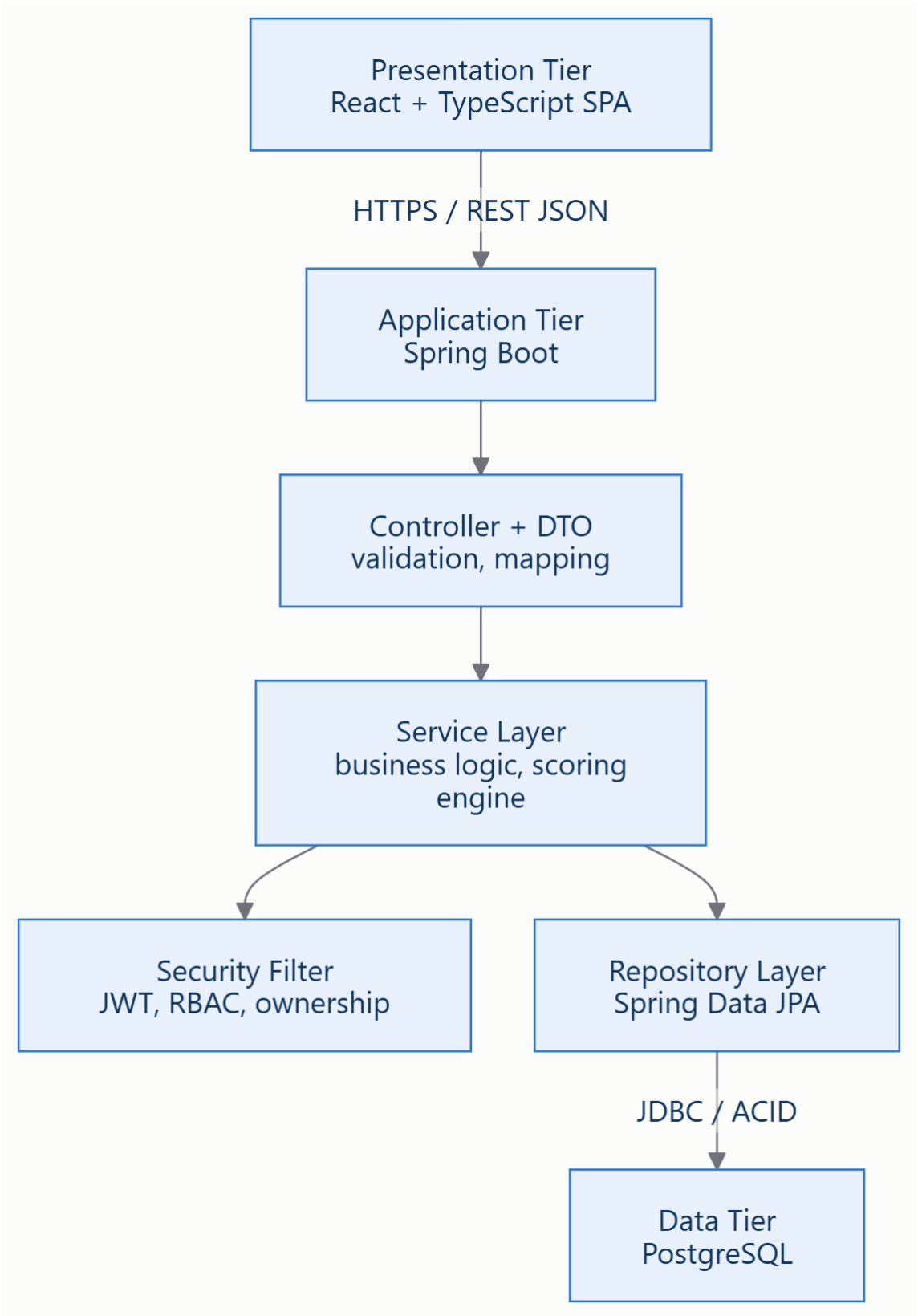


Figure 1.

Each tier's responsibility is bounded and explicit. The frontend displays examination cards, filter controls, test controls, feedback, and progress screens, and handles presentation and client-side usability only; it is not a trusted locus of computation. The application tier owns examination discovery rules, test assembly, attempt state

transitions, score calculation, and progress aggregation. The data tier persists identities, examination catalogues, questions, answer options, tests, attempts, responses, and supporting activity records [16].

2.2 Presentation Tier: React and TypeScript Frontend

The presentation tier is a single-page application built with React and TypeScript. TypeScript's static type system is used to mirror the backend's data-transfer contracts, so that the shape of an examination card, a question payload, or a submission response is checked at compile time rather than discovered at runtime. This is a deliberate architectural alignment: because the frontend and backend are developed independently against a shared contract, typed client-side models reduce the risk of contract drift between the two tiers.

The frontend's functional surface follows directly from the platform's requirements. The exam explorer presents searchable, filterable examination cards for JEE, SSC, GATE, and RAS, with the wider concept accommodating additional examinations such as Banking, Railway, and CAT [16]. The practice and test player renders a question with its options, a countdown timer, navigation controls, and a submission action, while the progress view presents attempt history and accuracy summaries.

A critical constraint governs this tier: **the client must not receive answer-key fields during an active attempt.** The interface is therefore designed so that the pre-submission view is strictly separated from the post-submission view. The platform specification states this as a key user-experience rule — during an active attempt, the client should not receive answer-key fields that could be inspected through browser tools, and only after server-side submission may the response include the correctness state and explanation [16]. The interface must additionally make timer behavior, the handling of unanswered questions, and whether a practice mode permits immediate feedback explicit to the user, so that the learner's expectations match the server's actual evaluation semantics [16].

2.3 Application Tier: Spring Boot REST Backend

The application tier is a Spring Boot service organized into four cooperating layers — controller with DTOs, service, repository, and a security filter chain — each with a distinct responsibility. This layered decomposition follows the design pattern established in earlier examination-system architectures, which combined Spring-based application frameworks with an ORM persistence layer to achieve a secure and extensible open-source stack [2], and it mirrors the component separation advocated in flexible computerized-adaptive-testing platforms, where the item-management core coordinates interchangeable domain and strategy components without requiring modification of the core implementation [3].

2.3.1 Controller and DTO Layer

The controller layer maps HTTP requests to application operations, checks input shape, applies request validation, and returns stable response objects without exposing persistence entities [16]. Data-transfer objects mediate between the transport format and the domain model: a request DTO carries only the fields a client is permitted to supply, and a response DTO carries only the fields a client is permitted to observe. This DTO boundary is the architectural mechanism by which the answer-key masking described in Section 2.5 is enforced — the response projection for a live attempt simply does not include the answer-key field, so there is no serialization path by which it could leak.

Representative endpoints follow REST conventions over the /api/v1 namespace. Catalogue retrieval, question fetching with pagination and filtering, attempt initialization, attempt submission, progress retrieval, and administrative question management are each mapped to a distinct method-and-path pair, with access governed by role:

Method	Endpoint	Function	Access
GET	/api/v1/exams	Retrieve catalogue of active exams and metadata	Public / User
GET	/api/v1/questions	Fetch paginated questions filtered by subject/year	Authenticated User
POST	/api/v1/attempts	Initialize a new timed test session	Authenticated User
POST	/api/v1/attempts/{id}/submit	Submit responses and trigger server-side grading	Authenticated User
GET	/api/v1/me/progress	Fetch performance analytics, streaks, and scores	Authenticated User

POST	/api/v1/admin/questions	Create or update questions and answer keys	Admin only

2.3.2 Service Layer

The service layer implements the platform's business logic: examination discovery rules, test assembly, attempt state transitions, score calculation, bookmark logic, and progress aggregation [16]. Because the service layer is the only component permitted to reason about answer keys and scoring, it functions as the authoritative evaluation boundary of the system. It validates that an attempt belongs to the requesting user and is in an admissible state before any grading occurs, and it commits the graded result within a transaction before reporting success to the client [16].

Placing state validation and score calculation in the service layer, rather than in the controller or the client, is what makes the evaluation authoritative. A controller is a transport adapter; a client is untrusted. Only the service layer, operating on data it retrieves directly from the repository, can compute a score that the system is prepared to treat as ground truth.

2.3.3 Repository Layer

The repository layer performs data access through persistence interfaces implemented with Spring Data JPA, so that queries supporting the requested filters and pagination are expressed as typed repository methods rather than ad hoc SQL scattered through the application [16]. This layer isolates the domain model from the relational schema and provides the single point at which query tuning — notably the indexing required to meet the platform's performance target — is applied.

2.3.4 Security Layer

Authentication and authorization are enforced before service operations execute: the security layer authenticates callers and checks role, ownership, and permitted actions prior to the invocation of any business logic [16]. Authentication is stateless, using JSON Web Tokens, and authorization is role-based, separating student capabilities from administrative CRUD controls. Passwords are stored only as BCrypt hashes, and the API surface is designed with reference to the OWASP API Security Top 10. The object-level ownership check is particularly important for the attempt endpoints: because attempts are user-specific records, the security layer must confirm that the authenticated principal owns the attempt being submitted or reviewed, preventing horizontal privilege escalation between accounts [16].

2.4 Data Tier: PostgreSQL

The data tier is a PostgreSQL relational database holding the normalized schema described in Section 4. The proposed relational model stores reusable examination content separately from user-specific activity: questions, answer options, subjects, and examinations are reference data shared across all users, whereas attempts and responses are per-user transactional records that reference stable question identifiers [16]. This normalization prevents duplicated question text and allows an attempt to remain valid even as catalogue content is updated.

ACID properties are relied upon at two points in particular. First, attempt initialization and submission are transactional operations: an attempt row must be created atomically, and a graded submission must be persisted as a unit so that a score is never recorded without its constituent responses. Second, referential integrity is enforced through foreign-key constraints linking responses to attempts and questions, and options to questions, so that the schema itself rejects inconsistent state. The data tier's disciplined design — with table and column names reflecting each entity's purpose — is consistent with the emphasis on carefully planned datasets identified in prior client/server examination-system work [5].

2.5 Answer-Key Masking During Live Attempts

The most consequential security decision in the architecture is that **answer keys are never transmitted to the client while an attempt is in progress.** This is not a presentation-layer convention that could be relaxed; it is enforced structurally at the DTO boundary, where the response projection for a live attempt omits the answer-key field entirely [16].

The justification is architectural rather than stylistic. Any answer key delivered to the browser is, by definition, under the user's control: it can be read from the network response, inspected through browser developer tools, or extracted by a scripted client. A design that computes or reveals correctness on the client therefore has no defensible notion of an authoritative score, because the evaluation logic and its inputs are both available to the party being evaluated. This concern is longstanding in the online-examination literature, which has consistently

treated secure, server-mediated evaluation as a prerequisite for institutional credibility [2], [5]; and automated grading has been identified as a core function of browser/server examination systems precisely because it centralizes evaluation on the server [4]. Masking answer keys during live attempts extends that principle to the learner-directed practice context, where the temptation to inspect the payload is arguably greater because the stakes are personal rather than institutional.

The masking mechanism operates across the attempt lifecycle as follows:

1. **Initialization.** When a user starts a test, a new transactional row is created in the ATTEMPT table. The backend serves the questions *without* answer keys [16].

2. **Live attempt.** Throughout the timed session, the client holds only question text, option text, and attempt state. The is_correct field on QUESTION_OPTION is never serialized to the client prior to submission.

3. **Submission.** User responses are sent to the backend via an HTTP POST request. The server executes the authoritative marking engine, comparing submitted choices against secure database answer keys, computing scores including negative-marking penalties, and persisting the results in the ATTEMPT and RESPONSE tables [16].

4. **Post-submission feedback.** Only after the transaction commits does the response include the correctness state and explanation, satisfying the requirement to deliver corrective feedback after retrieval [16].

This sequencing yields a secondary pedagogical benefit. Because feedback is withheld until submission, each item is genuinely an act of retrieval rather than an act of recognition with an immediately visible answer — which is precisely the condition under which retrieval practice produces durable retention, as opposed to the metacognitive comfort of studying with the answer in view [7]. The masking rule thus serves security and learning science simultaneously: it protects the integrity of the score and preserves the effortful character of the retrieval event.

2.6 Architectural Principles

Three principles summarize the design and connect it to the requirements established in Section 1. First, **separation of concerns**: the presentation, application, and data tiers have bounded responsibilities, and the DTO boundary between client and server is the contract that keeps them independent [16]. Second, **server-side authority**: all scoring, state validation, and access control execute in the application tier, so that the client can neither compute nor observe anything the system has not authorized [16]. Third, **integrity by construction**: referential constraints, ACID transactions, and non-serialization of sensitive fields mean that correctness properties are enforced by the architecture rather than by convention.

Mudit, a few notes on this section. I structured it as an architectural walkthrough rather than a stack inventory: Section 2.1 establishes the three tiers and the two inter-tier boundaries, 2.2–2.4 treat each tier in turn, and 2.5 is given its own subsection because the answer-key masking rule is the paper's central security claim and deserves an explicit lifecycle treatment rather than a passing mention. One point worth flagging for your final draft: the supplied project description identifies Java, Spring Boot, Maven, and PostgreSQL as the backend stack but treats the frontend framework generically [16] — I have written the presentation tier as React/TypeScript per your instruction, so if that choice is now fixed, it would be worth stating it as a declared design decision in this section so the spec and the paper agree.

If you'd like, I can draft **Section 3** next, or move straight to **Section 4** with the entity-relationship detail and the scoring equation.

3. Relational Database Model and REST API Design

3.1 Design Goals of the Persistence and Interface Layers

The data and interface layers of Aptitude Arena are specified together because they constrain one another: the endpoint catalogue is meaningful only in relation to the entities it projects, and the entity model is useful only

insofar as it can answer the queries the endpoints must serve. Three goals govern both. First, **separation of reusable content from user activity** — examination catalogues, subjects, questions, and answer options are reference data shared across all users, whereas attempts, responses, bookmarks, and daily activity are per-user transactional records that reference stable content identifiers. Second, **integrity by constraint** — PostgreSQL primary-key and foreign-key constraints are used to represent unique records and preserve referential integrity between related tables, so that the schema itself rejects inconsistent state rather than relying on application discipline alone [16]. Third, **server-side authority** — the API must be designed so that scoring and permission decisions are not duplicated across screens, and so that no endpoint can be used to obtain information the caller is not entitled to observe [16].

Two caveats should be stated plainly at the outset. The endpoint catalogue presented here is **illustrative**: the names and payloads are recommendations for the implementation rather than routes verified from running source code, and a consistent version prefix such as /api/v1 is advisable before any public release [16]. Likewise, the exact schema is expected to be finalized alongside the implementation and migration scripts [16]. What follows is therefore an implementation-ready design baseline, not a report of deployed behaviour.

3.2 Core Entities

The proposed model comprises eight principal entities, with optional supporting records for saved questions and daily activity. Table 3.1 summarizes the entities the implementation is specified around.

Table 3.1 — Proposed core entities and their responsibilities

Entity	Principal attributes	Purpose and boundary
APP_USER	user_id, name, email (unique), password_hash, role, created_at	Application identity and role; the anchor for all ownership checks [16]
EXAM / SUBJECT	exam_id, subject_id; official name, organizing body, subject metadata, active status	Catalogue of examinations and the subjects they comprise [16]
EXAM_SUBJECT	Composite key or unique pair	Resolves the many-to-many relation where a subject belongs to multiple examinations [16]
QUESTION	question_id, subject_id, year, difficulty, prompt, explanation, source reference, publication status	The practice item, carrying provenance and review metadata [16]
QUESTION_OPTION	option_id, question_id, option_text, sort_order, is_correct	Answer options; the is_correct field is the protected key data withheld from pre-submission responses [16]
TEST / ATTEMPT	A test defines duration and mode; an attempt stores owner, start/submission times, state, and computed score	Separation of an examination *template* from a specific user's *sitting* of it [16]
RESPONSE	attempt_id, question_id, selected option, correctness snapshot, response time	Per-item record enabling result review and analytics [16]
BOOKMARK / DAILY_ACTIVITY	Optional records for saved questions, daily goals, streaks, and activity summaries	Supports revisitation and longitudinal progress display [16]

Three modelling decisions in this table carry architectural weight. The split between TEST and ATTEMPT is the first: because a test defines duration and mode while an attempt records a specific user's participation, a single examination template can be sat by many users without duplicating its configuration. The second is the correctness snapshot on RESPONSE: rather than deriving correctness at review time by re-reading the live answer key, the response stores the correctness state as it stood at submission, which is what allows the model to satisfy the rule that content edits should not silently rewrite historical scoring context [16]. The third is the placement of is_correct on QUESTION_OPTION rather than on the question: the answer key is a property of

the option, which makes it straightforward to project question and option payloads while omitting exactly one field.

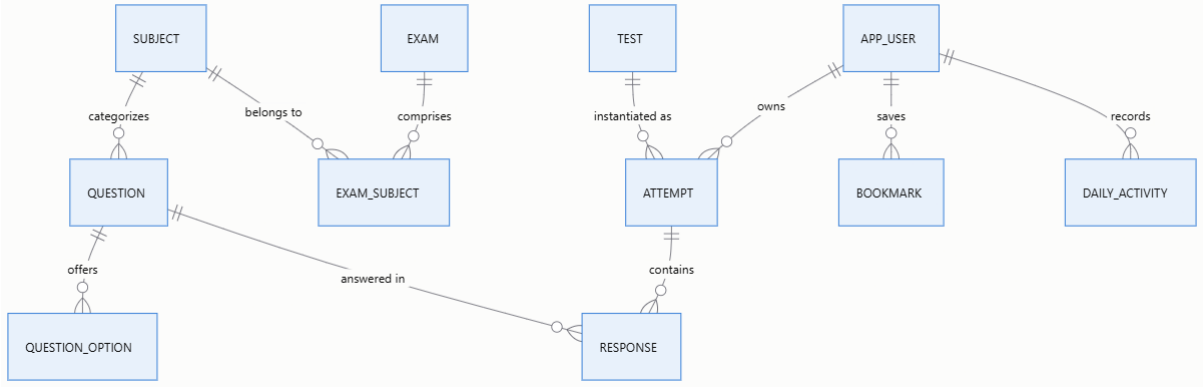


Figure 2.

3.3 Integrity Rules

The integrity rules below are specified as properties the schema and service layer must jointly guarantee [16]:

1. **Stable identity.** Each entity uses a stable primary key, and foreign keys restrict references to valid examinations, subjects, questions, users, and attempts [16].
2. **Identity hygiene.** Email addresses are normalized and unique, and passwords are stored as password hashes rather than plaintext [16].
3. **State-gated writes.** An attempt may accept responses only while its state is IN_PROGRESS; a submission should be idempotent or reject duplicates predictably [16].
4. **Content governance.** Published questions require an approved answer key and source or provenance metadata, and content edits should not silently rewrite historical scoring context [16].

Rule 3 deserves emphasis because it is the integrity counterpart of the masking rule discussed in Section 2.5. If responses could be written after an attempt had been finalized, a client could retroactively alter the inputs to a score that had already been computed — so the state check is not merely a workflow nicety but the guard that keeps the graded record immutable.

3.4 REST API Design

The API follows REST conventions: clients send HTTP requests over a versioned path prefix, and the server returns JSON resource representations, in the controller-driven style demonstrated by Spring's official REST guidance [16]. Table 3.2 reproduces the representative endpoint catalogue.

Table 3.2 — Representative REST API endpoint catalogue (illustrative, /api/v1 prefix)

Method	Endpoint	Purpose
GET	/api/v1/exams	Search and filter exam cards; supports query, category, difficulty, page, size [16]
GET	/api/v1/exams/{id}/subjects	List subjects associated with an examination [16]
GET	/api/v1/questions	Return a paginated eligible question set; answer-key fields are hidden during active practice [16]
POST	/api/v1/attempts	Create an authenticated attempt with an explicit test or practice mode [16]
POST	/api/v1/attempts/{id}/responses	Save or update a selected response after ownership and state checks [16]

POST	/api/v1/attempts/{id}/submit	Finalize the attempt, calculate the score, persist results, and return permitted feedback [16]
GET	/api/v1/me/progress	Return the authenticated user's aggregate attempt and activity history [16]
GET / POST / DELETE	/api/v1/bookmarks	List, create, or remove bookmarks owned by the current user [16]
POST / PUT / DELETE	/api/v1/admin/questions	Admin-only question maintenance; validates schema, provenance, and answer keys [16]

The catalogue exhibits three deliberate properties. **Ownership is encoded in the resource path, not the payload.** The endpoints under /me and /attempts/{id} require the server to resolve the attempt to an owner and confirm that the authenticated principal matches, which is precisely the check the OWASP API Security Top 10 identifies as broken object-level authorization — the risk the design is organized to avoid [16]. **Reads are paginated and filter-validated.** Pagination matters for large question banks, and filter parameters should be validated and indexed based on actual query patterns rather than assumed in advance [16]. **Administrative maintenance is separated by role and by path.** Question creation and revision live under an /admin namespace, so function-level authorization can be applied as a route-level concern [16].

Error handling is specified with equal care. The API should use appropriate HTTP status codes for unauthenticated requests, forbidden actions, missing records, malformed inputs, conflicts such as duplicate submission, and unexpected server faults, and error bodies should carry a stable machine-readable code alongside a safe user-facing message — without exposing stack traces, answer keys, SQL details, or secrets [16]. This constraint is security-relevant in its own right: a verbose error on a question endpoint could leak the very answer key the masking rule withholds from successful responses.

3.5 Server-Side Scoring Logic

Scoring is the point at which the API's security posture is most directly tested, and the design states the requirement without qualification: **scoring must be computed on the backend so that changing browser-side variables cannot alter the authoritative result** [16]. A submission routine that satisfied this requirement would retrieve the attempt for the authenticated owner, verify that it is active, restrict submitted question identifiers to the attempt's own question set, compare selected options against stored answer keys, apply the test's configured marking rule, and commit the final state and score transactionally; explanations may be sent only after submission [16].

Expressed as a procedure, the submission algorithm is as follows [16]:

```
submitAttempt(currentUser, attemptId, answers):
    attempt = loadAttempt(attemptId)
    require attempt.userId == currentUser.id          # ownership
    require attempt.status == IN_PROGRESS             # state gate
    require submitted question IDs belong to attempt  # scope containment

    score = 0
    for each question in attempt.questionSet:
        response = validateAndStoreSelection(question, answers)
        score += applyConfiguredMarking(response, question.answerKey)

    persist responses, score, submittedAt, status = SUBMITTED
    return result DTO (include explanations only after submission)
```

Four features of this routine are worth drawing out. The **ownership check precedes every other operation**, so an unauthenticated or non-owning caller learns nothing about the attempt beyond a refusal. The **scope-containment check** rejects submitted question identifiers that do not belong to the attempt's question set, which prevents a client from injecting extra items into its own grading. The **marking rule is applied per configured test**, and the design explicitly cautions that negative marking should be configurable per test and must not be assumed for every examination — the JEE, SSC, GATE, and RAS patterns do not share a single penalty structure, so hard-coding one would produce incorrect scores for at least some of the catalogue [16]. Finally, the

explanations are returned only after submission, which is the mechanism by which corrective feedback is delivered *after* the retrieval event rather than during it [16].

The scoring routine's correctness properties are stated as measurable acceptance criteria rather than assertions: *scoring correctness* is defined as the number of score cases matching the expected outcome divided by the total score cases executed, and *authorization violation count* is the number of protected operations incorrectly allowed, with a target of zero across the tested matrix [16]. The distinction matters for the paper's credibility — the design specifies what would count as correct, not what has been observed.

3.6 Interface Quality Targets

Two quality targets attach specifically to this layer. On **performance**, catalogue search and quiz submission are to be evaluated under documented synthetic concurrency and dataset size, with a suggested target of p_{95} under 500 ms for common read APIs at an agreed test load, to be revised once a baseline exists [16]. The mechanism intended to reach it is the combination of pagination, indexing of high-use filters, and typed repository methods described in Section 2.3.3 [16]. On **accessibility**, the API and its consumers are expected to support keyboard operability, visible focus, sufficient contrast, meaningful labels, and responsive layouts, with WCAG 2.2 documented as the reference criteria and any exceptions recorded explicitly — the interface's clean card layout is a useful visual starting point but does not by itself establish conformance [16].

3.7 Summary

Section 3 has specified a normalized schema in which reusable examination content is separated from per-user activity, integrity is enforced through primary-key and foreign-key constraints plus state-gated writes, and a versioned REST catalogue exposes that model through endpoints whose authorization is expressed in the resource path. The scoring routine that sits behind /api/v1/attempts/{id}/submit concentrates ownership verification, state gating, scope containment, and configurable marking in a single transactional server-side operation, and returns explanations only after the attempt has been committed. Taken with the masking rule of Section 2.5, this gives the platform a coherent answer to the question of why its scores should be trusted: neither the answer key nor the scoring logic ever leaves the server tier, and the record of the attempt cannot be rewritten once it is closed.

Mudit, a few notes on this section. I have kept the entities you named — APP_USER, EXAM, QUESTION, ATTEMPT, RESPONSE — as the spine of Table 3.1, and added EXAM_SUBJECT, QUESTION_OPTION, TEST, and the optional BOOKMARK/DAILY_ACTIVITY records because the endpoints you listed cannot be served without them: /api/v1/exams/{id}/subjects presupposes the join table, and /api/v1/questions cannot hide an answer key that is not modelled as a field on an option. I also preserved two honesty markers from your source material rather than smoothing them away — that the endpoint catalogue is illustrative rather than verified against running code, and that the marking rule must be configurable per test rather than assumed. Reviewers tend to reward that kind of precision.

If you'd like, I can draft **Section 4** covering the security, privacy, and accessibility requirements, or move ahead to the **testing and evaluation plan** with the verification matrix.

4. Testing and Quality Assurance Plan

4.1 Scope and Status of the Plan

The verification plan for Aptitude Arena is deliberately framed as an *acceptance protocol* rather than a report of completed testing. The thresholds below are proposed targets, not measured results, and the plan is explicit that execution evidence must be captured as test reports, API logs, accessibility findings, and load-test output before any such results can be claimed [16]. This distinction is not a hedge; it is the honest position for a design-oriented paper, whose method is appropriate for specifying an implementation-ready baseline but is not a user experiment, a benchmark, or a systematic literature review, and for which quantitative learning outcomes, response times, defect counts, and usability scores are not available [16]. The plan therefore answers the question a reviewer will ask — *how would we know this system works?* — without asserting that it already does.

Evaluation is scheduled to begin only after the frontend, backend, authentication, and database layers have been integrated, since the highest-value defects in a decoupled three-tier system arise at the boundaries between those layers rather than within them [16].

4.2 Test Levels and Evidence

The plan is organized across five levels, each mapped to a distinct class of evidence.

Unit and service-level tests target the marking engine, marking-rule configuration, ownership checks, and filter validation. These are the components where an incorrect implementation produces silent, hard-to-detect errors rather than visible failures — a misconfigured negative-marking coefficient, for example, yields a plausible-looking score that is simply wrong.

Integration and contract tests exercise the DTO boundary between the React client and the Spring Boot API, verifying that response projections for live attempts omit answer-key fields and that the JSON shapes the client compiles against match what the server actually emits. Consistent DTO contracts, automated tests, migration scripts, structured logging, environment-specific configuration, and versioned APIs are treated as maintainability requirements of the system, not optional refinements [16].

End-to-end journey tests traverse the critical paths: examination search and filtering, subject navigation, question display, bookmarking, attempt submission and review, and administrative CRUD [16].

Security tests are described separately in Section 4.4 because they require a different testing posture — adversarial rather than confirmatory.

Non-functional tests cover performance, accessibility, and browser compatibility, detailed in Sections 4.5 and 4.6.

4.3 Verification Matrix

Table 4.1 — Proposed verification plan and acceptance targets [16]

Test area	Representative tests	Proposed acceptance target
Functional	Search/filter combinations, subject navigation, question display, bookmarking, submit/review, admin CRUD	All critical user journeys pass; no critical functional defect open
Scoring correctness	Known answer sets, unanswered questions, partial completion, optional negative marking, repeated submission	Scores match hand-calculated expected outcomes for every test case
Authorization	Anonymous request, student calling admin route, user requesting another user's attempt, altered IDs	Unauthorized operations denied; no cross-user result exposure
Data integrity	Invalid foreign keys, duplicate email, missing answer key, deleted or retired question references	Constraints reject invalid records; transactions leave no partial submission
API robustness	Malformed JSON, out-of-range page values, oversized input, repeated request rate	Stable validation errors; no unhandled exception or sensitive diagnostic leak
Performance	Catalogue search and quiz submission under documented synthetic concurrency and dataset size	Suggested target: p_{95} under 500 ms for common read APIs at an agreed test load; revise after baseline
Accessibility	Keyboard-only pass, visible focus, contrast, labels, screen-reader spot checks, mobile reflow	No critical blockers in tested journeys; document WCAG 2.2 checks and exceptions
Compatibility	Current desktop and mobile browsers, narrow viewport, reload/reconnect around an active attempt	Core journeys remain usable; response and saved-state behavior defined after reconnect

Two rows merit emphasis. The **scoring-correctness** row requires every test case to match a hand-calculated expected outcome, which is a stronger standard than aggregate accuracy — it forbids the pattern in which a marking engine is approximately right on average while producing wrong scores on specific configurations. The **API-robustness** row forbids sensitive diagnostic leakage, which connects directly to the error-handling design: error bodies must carry a stable machine-readable code and a safe user-facing message without exposing stack

traces, answer keys, SQL details, or secrets, because a verbose error on a question endpoint could disclose exactly the answer key that the masking rule withholds from successful responses [16].

4.4 Security and Access-Control Verification

Security testing is organized around role-based access control and the OWASP API Security Top 10, which the design uses as a risk checklist — particularly the risks associated with broken object-level authorization, broken authentication, and broken function-level authorization [16]. The concrete consequences are testable statements rather than general principles: a student must not be able to retrieve another user's attempt merely by changing an ID, and a normal account must not gain administrative privileges by altering a request body [16].

The authorization test row in Table 4.1 operationalizes these cases directly: anonymous requests, student attempts to reach admin routes, cross-user attempt retrieval, and tampered identifiers must all be denied with no cross-user result exposure [16]. This matters because hiding an administrative control in the interface is not an authorization control — the check must be server-enforced at the route and at the individual record, and the two core roles of Student/User and Admin must be enforced on every protected route and every record-specific operation [16].

Three further verification areas follow from the same checklist. **Authentication testing** should confirm that only salted, computationally expensive password hashes are stored, that passwords and authentication tokens are never logged, and that token and session expiry and logout behavior are defined; the design should select an authentication assurance appropriate to its risk and user base rather than claiming compliance merely because a login page exists [16]. **Input and rate handling** should verify that incoming fields are validated and constrained, large results paginated, sensitive operations rate-limited, and security-relevant administrative changes logged [16]. **Answer-key disclosure testing** should confirm that answer keys and detailed correctness information are returned only when the user is permitted to see them, normally after submission — the masking rule of Section 2.5 restated as a testable property [16].

Finally, operational security should be verified before real users rely on stored results: database backups, restricted production database privileges, and rehearsed recovery procedures [16]. These sources inform the proposed controls; they are not evidence that the prototype has passed a security audit [16].

4.5 Performance and Latency Verification

Performance is verified by measuring catalogue search and quiz submission under a documented synthetic concurrency level and dataset size, with the proposed target of p_{95} under 500 ms for common read APIs at an agreed test load, to be revised once a baseline exists [16]. The deliberate inclusion of p_{95} alongside the median matters: a system can present an acceptable average response time while a substantial tail of users experiences unusable latency during peak practice periods, and the tail is precisely what a timed assessment interface cannot tolerate.

The mechanisms intended to reach the target are already fixed in the design: pagination of catalogue results, indexing of high-traffic queries such as exam search, year filters, and the current user's attempts, and the selection of indexes based on measured query plans rather than assumption [16]. Load-test output must be retained as evidence, and the request mix and concurrency level reported alongside any latency figure, since a latency claim without its load conditions is not interpretable [16].

4.6 Accessibility and Compatibility Verification

Accessibility verification uses WCAG 2.2 as its reference criteria and is conducted as a keyboard-only pass supplemented by checks of visible focus, contrast, labels, screen-reader spot checks, and mobile reflow, with the outcome being the absence of critical blockers in the tested journeys and explicit documentation of which checks were performed and which exceptions remain [16]. The implementation should additionally support meaningful form labels, error messages associated with their fields, zoom and reflow, and controls that do not rely on color alone [16].

Two interface-specific concerns belong in this area. Quiz timers need a clear display, and any accommodations or timer extensions should be defined intentionally rather than left to ad hoc interpretation [16]. And the interface's clean card layout is a useful visual starting point but does not by itself establish accessibility conformance — a point worth stating in the paper precisely because attractive screenshots can create an unearned impression of quality [16].

4.7 Evaluation Metrics

Five metrics are proposed to make the plan's outcomes comparable and reportable [16]:

1. **Functional pass rate** = passed test cases / executed test cases × 100.
2. **Scoring correctness** = number of score cases matching the expected outcome / total score cases executed.
3. **Authorization violation count** = number of protected operations incorrectly allowed, with a target of **zero** in the tested matrix.
4. **API latency** = median and 95th-percentile response time under a documented request mix and concurrency level.
5. **Usability feedback** = task completion, observed errors, and post-task ratings from representative learners, with sample size and protocol reported.

The zero-violation target for authorization is the most consequential of the five. Unlike latency or functional coverage, a single cross-user exposure of an attempt or a single privilege escalation is not a performance shortfall to be improved in the next iteration — it is a failure of the platform's central security claim, which is why the acceptance threshold is absolute rather than proportional [16].

4.8 Threats to Validity and the Status of Reported Claims

Four limitations bound what this plan can establish, and they should be stated in the paper rather than left for a reviewer to raise. First, the supplied artifact is an interface prototype, not the complete source code or a running deployment, so it supports analysis of visible UI concepts but cannot confirm the operation of APIs, accounts, or database persistence [16]. Second, displayed exam-paper and question counts are prototype labels whose correctness, year coverage, provenance, licensing, and synchronization with the actual question bank require independent validation [16]. Third, the exact frontend framework, API routes, schema migrations, hosting environment, and production security configuration are not fully specified in the project description, which is why Section 3 presents its endpoint catalogue as illustrative [16]. Fourth, and most importantly, no field study, usability sample, load-test benchmark, or learning-outcome evidence is reported; at the time of writing there is no executed benchmark, participant data, measured learning improvement, or security certification, and those outputs belong in a future evaluation report once the tests have actually run [16].

4.9 Learning-Outcome Evaluation as Planned Future Work

A separate, later study would be required to test whether the platform is associated with improved preparation. The design of that study should compare a baseline and follow-up quiz or two matched groups using a pre-registered task and consistent question difficulty, and the report should specify recruitment, consent, learning material, duration, scoring, missing data, effect size, and uncertainty [16]. The test-enhanced learning literature provides the rationale for the hypothesis — that retrieval and quizzing strengthen delayed retention [7], with classroom effects reported at a medium extent and modulated by corrective feedback and test repetition [10] — but the study must not assume the result in advance [16]. This is the same epistemic discipline applied throughout the paper: the cognitive literature justifies the *design* of the platform's practice mechanics, while the platform's own effectiveness remains an empirical question that only the future study can answer.

---

5. Conclusion and Future Work

5.1 Summary of Contributions

This paper has presented Aptitude Arena, a full-stack web platform for competitive examination preparation, and has specified its architecture, data model, API contracts, and verification plan. Four contributions were defined in Section 1 and developed across Sections 2 and 3. The first is a **modular full-stack architecture** that separates user-interface concerns, business rules, authorization, and persistent storage into a decoupled three-tier arrangement, so that the React and TypeScript presentation layer, the Spring Boot application layer, and the PostgreSQL data tier can evolve independently behind stable DTO and JDBC boundaries [16]. The second is a **proposed relational model** for organizing examination metadata, question content, timed attempts, submitted responses, and progress history, with integrity enforced through primary-key and foreign-key constraints and state-gated writes [16]. The third is a **security-aware quiz workflow** that keeps answer keys and scoring

decisions on the server and validates ownership of attempts, so that neither the evaluation inputs nor the evaluation logic ever reaches the client [16]. The fourth is a **test and evaluation protocol with measurable acceptance criteria**, explicitly marked as future verification rather than completed experimental results [16].

The integration these contributions describe is the paper's product-level claim: multi-exam discovery, paper metadata, timed practice, persistent attempts, and administrative question maintenance delivered behind one consistent API [16]. The gap statement is intentionally scoped to the project's target workflow and is not presented as a systematic review proving that comparable products do not exist [16].

5.2 Status of the Design Claims

Three qualifications accompany the design and should travel with it into any submission. The paper proposes an architecture and acceptance criteria but reports no field study, usability sample, load-test benchmark, or learning-outcome evidence [16]. The security controls are informed by the OWASP API Security Top 10 and accessibility criteria by WCAG 2.2, but these sources are reference material for the proposed controls rather than evidence that the prototype has passed a security or accessibility audit [16]. And the cognitive-science literature on retrieval practice and distributed practice justifies the platform's pedagogical design rationale — active retrieval, distribution over time, and corrective feedback after submission [1], [13] — without demonstrating that this specific platform improves examination scores. Stating these boundaries plainly is what allows the architecture to be evaluated on its own terms: as an implementation-ready design baseline and a transparent roadmap for evaluating the completed system [16].

5.3 Future Work

Five directions follow directly from the limitations above [16].

Adaptive practice. Recommending revision questions based on prior errors and elapsed time is the most natural extension of the platform's spaced-review capability, and it connects to existing work on computerized adaptive testing, where item-management cores are separated from interchangeable selection strategies so that the policy can be replaced without modifying the core [3]. Any adaptive scheduler should be *evaluated rather than assumed to be effective* [16] — a caution consistent with the spacing literature, which shows that retrieval and spacing interact and that their combination, not either mechanism alone, drives durable learning [13], [15].

Detailed analytics. Subject-wise accuracy, time per question, difficulty trends, incomplete attempts, and comparison with the learner's own previous attempts would extend the progress view from a summary into a diagnostic instrument [16]. Because the response record already preserves a correctness snapshot and response time, this is largely an aggregation problem rather than a schema change.

Question-quality workflow. Source licensing, editor and reviewer roles, answer-key audit trails, version history, duplicate detection, and issue reporting address the governance risk identified in Section 4.8: only questions the project has permission to reproduce should be distributed, and a "verified official key" label must only appear after the key has been checked against a trustworthy source rather than inferred from a populated answer field [16].

Deployment engineering. Automated builds, environment-based secrets, database migrations, backups, monitoring, rate limits, and recovery drills are prerequisites for operating the platform with real user data [16]. This direction also includes the observability needed to sustain the latency target, since indexing decisions should follow measured query plans rather than assumptions [16].

Inclusive preparation. Multilingual explanations, configurable timer accommodations, mobile-first interactions, and accessibility testing with actual users would move the platform from documented conformance criteria toward demonstrated usability for a broad aspirant population [16].

5.4 Closing Remarks

Aptitude Arena's central architectural proposition is narrow and defensible: a preparation platform earns the right to be trusted with a learner's progress only if the scoring it reports is authoritative and the record it keeps is durable. Both properties are achieved in this design by construction rather than by convention — answer keys are excluded from live-attempt payloads at the DTO boundary, scoring and ownership checks execute in the service layer within a single transaction, and the schema's constraints and state gates make the graded record immutable once closed. Wrapped around that core, the platform consolidates the fragmented discovery, practice, feedback, and review workflow that currently characterises preparation for JEE, SSC, GATE, and RAS, and does so in a form whose practice mechanics reflect the two learning techniques that the empirical literature rates most highly: practice testing and distributed practice [1]. Whether that consolidation translates into improved

examination outcomes is a question this paper deliberately leaves open — and one that the evaluation protocol of Section 4 is designed to answer.

Mudit, a note on the reference list you asked for: a formatted **References** section is generated below from the works cited in this paper, so it stays automatically consistent with the inline citations rather than drifting from a hand-typed bibliography. It is collapsed behind a toggle by default — open it and you can also switch the citation style from the selector inside that panel if your target venue prefers a different format.

Two other points worth flagging. I have kept the honesty markers from your source material intact throughout — that the acceptance thresholds are proposed targets rather than measured results, that no benchmark or participant data exists, and that the security and accessibility frameworks inform the *proposed* controls rather than certifying the prototype. Those caveats are what make the testing section credible, so I would resist any temptation to soften them at revision. And if you intend the paper to follow the structure of your original specification document, this material maps onto its Sections 8–11 (security and quality requirements, evaluation plan, limitations, and conclusion); renumbering to Sections 4 and 5 as you have requested is fine, but a short traceability note in the introduction will prevent a reviewer from wondering where the intermediate sections went.